

Poursuite de robots

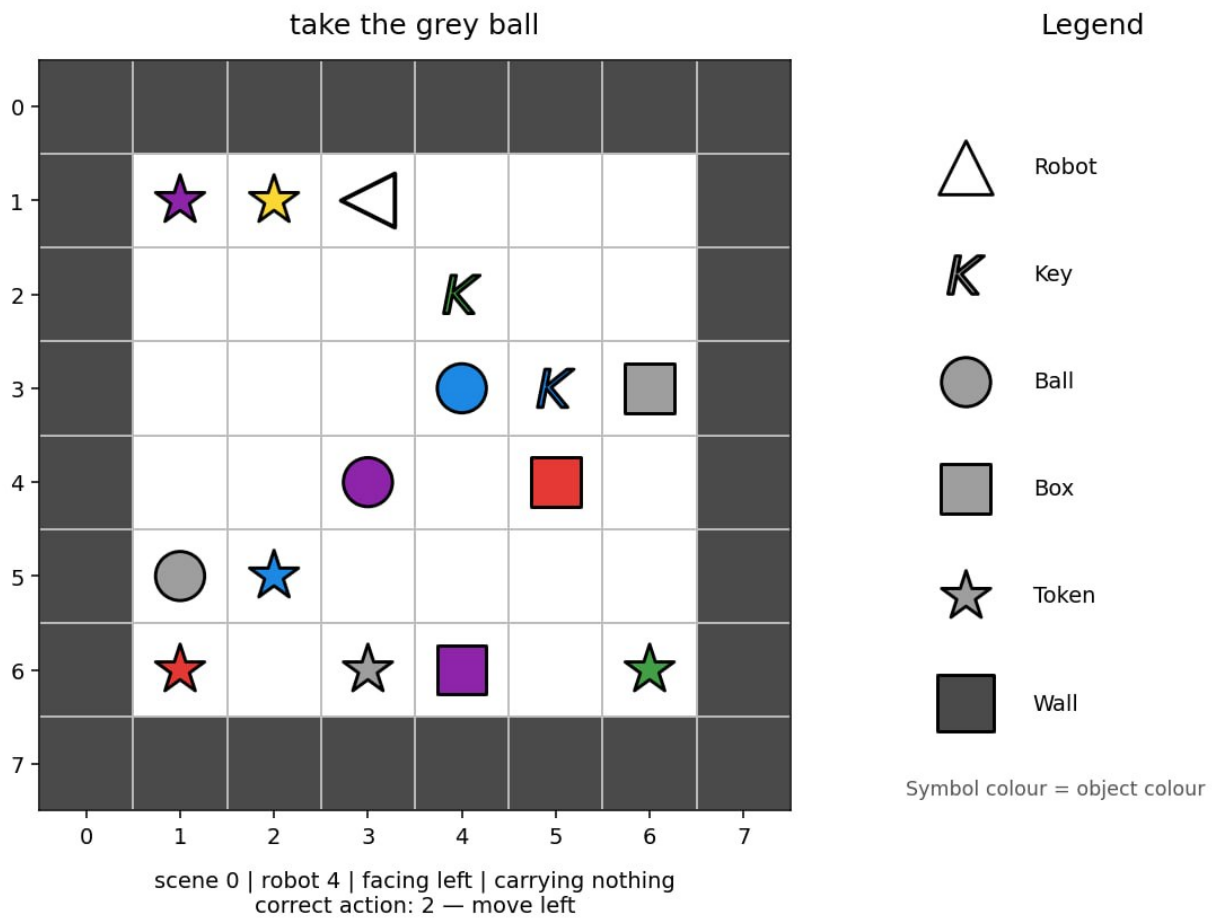
- **Limite de temps** : 5 minutes
- **Environnement** : un GPU (≈ 16 GB VRAM), sans internet
- **Taille de la solution** : `solution.ipynb` ≤ 1 MB
- **Stockage** : 5 GB

Tâche

Il y a six robots. Chaque robot évolue dans une petite pièce représentée par une grille. Chaque pièce possède une zone jouable 6×6 entourée de murs, de sorte que le tableau `image` complet est de taille 8×8 (zone jouable + murs).

Chaque robot reçoit une instruction en anglais décrivant une tâche. Une capture d'écran (ou *snapshot*) peut être pris à n'importe quel moment pendant que le robot exécute la tâche. Votre objectif est de prédire la prochaine action du robot.

Les robots ne suivent pas toujours le chemin le plus court. Le Robot 0 peut se comporter différemment du Robot 1, mais chaque robot suit son propre comportement de manière cohérente. Utilisez les exemples d'entraînement, qui comprennent les prochaines actions correctes, pour apprendre ces comportements.



Il existe trois types de missions :

- **aller jusqu'à (*go to*)** un objet, par exemple "approach the red ball" ;
- **ramasser (*pick up*)** un objet, par exemple "grab the blue key" ;
- **placer un objet à côté d'un autre (*put one object next to another*)**, par exemple "place the red box beside the green ball".

Une même instruction peut être formulée de plusieurs manières. La base de test peut contenir de nouvelles combinaisons de formulations, de couleurs et de types d'objets connus. Cependant, chaque mot, modèle de formulation, couleur, type d'objet et type de mission utilisé dans la base de test apparaît également dans la base d'entraînement.

Chaque échantillon comporte les champs suivants :

Champ	Signification
robot_id	lequel des 6 robots il s'agit (0-5)
image	la pièce, un tableau d'entiers $8 \times 8 \times 2$ dans lequel le canal 0 contient la valeur catégorielle object_idx (par exemple, 1=case vide, 2=mur, 10=robot) et le canal 1 contient la valeur catégorielle colour_idx (0-5).
direction	la direction vers laquelle le robot est actuellement orienté
mission	l'instruction visible en langage naturel
carrying	null ou [object_idx, colour_idx] pour l'objet transporté

Les lignes sont des captures d'écran (ou *snapshot*) indépendants, présentés dans un ordre aléatoire. Elles ne constituent pas des épisodes, et aucune observation ou action antérieure n'est disponible au moment de l'évaluation.

Le `visualize_dataset.ipynb` fourni vous permet d'examiner les observations dont dispose le modèle dans différentes situations.

Encodage de la grille

`image[row][column] = [object_idx, colour_idx]`. Le premier indice correspond à la ligne, de haut en bas, et le second à la colonne, de gauche à droite. Le tableau comprend la bordure extérieure constituée de murs, de sorte que l'intérieur navigable est 6×6 .

Identifiants des objets :

id	objet
1	case vide (empty cell)
2	mur (wall)
5	clé (key)
6	balle (ball)
7	boîte (box)
10	robot (robot)
11	jeton (token)

Des jetons peuvent apparaître dans la pièce, mais ils ne sont jamais mentionnés dans les missions.

Les identifiants des couleurs sont 0 rouge, 1 vert, 2 bleu, 3 violet, 4 jaune et 5 gris. Le canal des couleurs n'a aucune signification pour les cases vides et les murs.

L'image ne comporte que les deux canaux ci-dessus. La direction du robot est fournie une seule fois, dans le champ `direction` de premier niveau ; elle n'est pas dupliquée dans `image`.

Actions

Pour les codes 0-3, les actions de déplacement utilisent la correspondance absolue suivante :

action	signification
0	se déplacer vers le haut (move up)
1	se déplacer vers le bas (move down)
2	se déplacer vers la gauche (move left)
3	se déplacer vers la droite (move right)
4	ramasser (pick up)
5	déposer (drop)

Le champ `direction` indique l'orientation actuelle selon la convention suivante : 0 = Haut (ligne - 1) ou Up (row - 1), 1 = Bas (ligne + 1) ou Down (row + 1), 2 = Gauche (colonne - 1) ou Left (col - 1), 3 = Droite (colonne + 1) ou Right (col + 1).

Une action de déplacement oriente d'abord le robot dans cette direction absolue, puis tente de le déplacer d'une case. Un mur ou un objet peut bloquer le déplacement, mais la direction change tout de même. `pick up` et `drop` agissent exclusivement sur la case cible adjacente définie par la direction (par exemple, si `direction=0`, l'action porte sur (ligne - 1, colonne) ou (row - 1, col)).

Dataset

Vous recevez deux dossiers :

Dossier	Lignes	labels.json ?	Utilisez le pour
dataset/train/	60,000	inclus	entraîner votre modèle
dataset/test_public/	3,600	inclus mais disponible après soumission	exécuter et évaluer vous-même votre pipeline

Chaque dossier contient `observations.json`, une liste JSON des échantillons décrits ci-dessus. `labels.json` est une liste JSON alignée d'actions (0-5).

La base d'entraînement contient exactement 10,000 lignes par robot et 20,000 lignes de chaque famille de tâches. La base de test public contient 600 lignes par robot. Encapsulez `image` avec `numpy.asarray(...)` si vous avez besoin d'un tableau.

Au moment de l'évaluation, `dataset/test_public/` est remplacé de manière transparente par un ensemble caché de 3,600 observations au même format, mais sans `labels.json`. Le classement public utilise `test_leaderboard_a` ; le classement final utilise `test_leaderboard_b`. **Un notebook qui lit systématiquement les étiquettes de test échouera. Lisez les étiquettes uniquement depuis `dataset/train/`.**

NB : _Pas de modèles pré-entraînés disponibles pour ce challenge.

Sortie

Écrivez `predictions.json` dans le répertoire de travail du notebook. Il doit s'agir d'une liste JSON contenant une action entière (0-5) pour chaque ligne de `dataset/test_public/observations.json`, dans le même ordre. Pour une base de test hypothétique contenant six échantillons, une sortie valide serait :

```
[0, 3, 2, 2, 5, 4]
```

Un fichier JSON manquant ou invalide, un nombre incorrect de prédictions, une valeur non entière, ou une action en dehors de {0,1,2,3,4,5} est rejeté sans recevoir de score.

Évaluation

Le score est l'**exactitude (*accuracy*) moyenne par robot** sur une échelle de 0-100. L'exactitude (*accuracy*) est d'abord calculée indépendamment pour chaque robot, puis moyennée sur les six robots. Chaque robot a donc le même poids.

Comment soumettre

1. Ouvrez le fichier `solution.ipynb` et exécutez toutes les cellules.
2. Vérifiez qu'il écrit `predictions.json` avec 3,600 prédictions pour la base de test public.
3. Améliorez le modèle si vous le souhaitez ; la baseline fournie ne fait que montrer le format d'entrée et de sortie requis.
4. Dans l'onglet Git de JupyterLab, ajoutez et commitez `solution.ipynb`, puis poussez-le.
5. Retournez à la page du *challenge* et cliquez sur **Soumettre**.

Soumettez exactement un fichier nommé `solution.ipynb`.